

Prob 1. (60 pts) Assume the address of a 16-element `uint8_t` array `my_arr` is `0x2000_0100`, and the values are given in the table below.

Address	Byte 0	Byte 1
0x2000_0100:	0x00	0x02
0x2000_0102:	0x04	0x06
0x2000_0104:	0x08	0x0A
0x2000_0106:	0x0C	0x0E
0x2000_0108:	0x10	0x12
0x2000_010A:	0x14	0x16

Assume we run the following code to print out some values.

```

1  uint8_t *p8a = (uint8_t *)0x20000100;
2  uint8_t *p8b = (uint8_t *)0x2000010C;
3
4  *p8b++ = *(p8a + 2);
5  *p8b++ = +++(p8a + 4);
6  *p8b++ = +++p8a++;
7  *p8b++ = *p8a++;

```

Determine

- (10 pts) The addresses and values that have been changed by the right side expressions of the statements in lines 4 to 7. For example, if we run `*p8b = ++++p8a` as the first line of runnable code, then, we would have the value at address `0x2000_0101` changed from 2 to 3.
- (20 pts) The four 8-bit values in the memory starting at address `0x2000_010C` after running the above code.
- (30 pts, 5 pts for lines 4 and 7 each and 10 pts for lines 5 and 6 each) Write the code in lines 4 to 7 in an assembly function with the following prototype and call instance:

```

1  void my_fn(uint8_t *pa, uint8_t *pb);
2  my_fn(p8a, p8b);

```

Soln:

Part 1.

`*p8b++ = +++(p8a + 4);` will change the content at `0x2000_0104` from 8 to 9. `*p8b++ = +++p8a++;` will change the content at `0x2000_0100` from 0 to 1.

Part 2.

- `0x2000_010C`: 4
- `0x2000_010D`: 9

- 0x2000_010E: 1
- 0x2000_010F: 2

Part 3.

```

1  @ *p8b++ = *(p8a + 2);
2  @ *p8b++ = ++*(p8a + 4);
3  @ *p8b++ = ++*p8a++;
4  @ *p8b++ = *p8a++;
5
6  @ void my_fn(uint8_t *pa, uint8_t *pb);
7  @   Input:
8  @     * pa -> r0
9  @     * pb -> r1
10 @   Output:
11 @     * none
12 @   Others:
13 @     * temp -> r2
14
15 my_fn:
16     ldrb    r2, [r0, #2]
17     strb    r2, [r1], #1
18
19     ldrb    r2, [r0, #4]
20     add     r2, #1
21     strb    r2, [r1], #1
22     strb    r2, [r0, #4]
23
24     ldrb    r2, [r0]
25     add     r2, #1
26     strb    r2, [r1], #1
27     strb    r2, [r0], #1
28
29     ldrb    r2, [r0], #1
30     strb    r2, [r1], #1
31
32     bx     lr

```

:-

Prob 2. (74 pts) Assume we are given the following code snippets:

Part A: C code:

```

1  // Prototype of functions defined in a .s file
2  extern void task1(uint8_t *pIpt, int32_t *pOup);
3  extern void task2(uint8_t *pIpt, int32_t *pOup);
4  extern void task3(uint8_t *pIpt, int32_t *pOup);
5  extern void task4(uint8_t *pIpt, int32_t *pOup, int i);
6
7  // Global variable
8  uint8_t ipt[16];    // Input data
9  int32_t opt[4];     // Output data
10
11 int main(void) {
12     for (int i = 0; i < 16; i++) {
13         ipt[i] = i<<4;

```

```

14     }
15
16     // Task 1.
17     task1(ipt, opt);
18     printf("Out1 = 0x%X, Out2 = 0x%X\n", opt[0], opt[1]);
19
20     // Task 2.
21     task2(ipt, opt);
22     printf("Out1 = 0x%X, Out2 = 0x%X\n", opt[0], opt[1]);
23
24     // Task 3.
25     task3(ipt, opt);
26     printf("Out1 = 0x%X, Out2 = 0x%X\n", opt[0], opt[1]);
27
28     // Task 4.
29     task4(ipt, opt, 1);
30     printf("Out1 = 0x%X, Out2 = 0x%X\n", opt[0], opt[1]);
31
32     while (1);
33 }

```

Part B: assembly code:

```

1  task1:
2      ldrd    r2, r3, [r0, #8]!
3      strd    r2, r3, [r1, #0]
4      bx     lr
5
6  task2:
7      ldrsb   r2, [r0, #4]!
8      ldrsb   r3, [r0, #4]!
9      strd    r2, r3, [r1, #0]
10     bx     lr
11
12 task3:
13     ldrsh   r2, [r0], #10
14     ldrsh   r3, [r0], #4
15     strd    r2, r3, [r1, #0]
16     bx     lr
17
18 task4:
19     ldrh    r3, [r0, r2, LSL #1]
20     str     r3, [r1, #0]
21     add     r2, #1
22     ldrh    r3, [r0, r2, LSL #1]
23     str     r3, [r1, #4]
24     add     r2, #1
25     bx     lr
26
27     .end

```

Answer the following questions (without running the program first):

- (8 pts) What will be the printout after Task 1?
- (14 pts) What will be the printout after Task 2 and what will be the values in `r0` and `r1` in hexadecimal after running Task 2 assuming their values are 0x20001010 and 0x20001040, respectively, before running

the first line of code in function Task2?

- (14 pts) What will be the printout after Task 3 and what will be the values in `r0` and `r1` in hexadecimal after running Task 3 assuming their values are 0x20001010 and 0x20001040, respectively, before running the first line of code in function Task3?
- (14 pts) What will be the printout after Task 4 and what will be the values in `r2` and `r3` in hexadecimal after running Task 4?
- (24 pts) What will be the corresponding C code for each task if all the tasks are listed in the same function? We have two notes here, as illustrated in the code below:
 - If all the tasks are listed in the same function, we cannot change the values of `ipt` and `opt` in each task; otherwise, the results will be totally different. To this end, we need to define new pointers and work on these new pointers.
 - You don't have to write the C code with data in the 64-bit format. As long as we have the same result, it will be ok.

```
1 ldr    r2, [r0, #8]
2 ldr    r3, [r0, #12]
3 strd   r2, r3, [r1, #8]!
```

Then, we can write the C code in two different ways, both are good:

```
1 int32_t *iptr = (int32_t *)ipt;
2 int32_t *optr = (int32_t *)opt;
3 optr += 2;
4 *optr = *(iptr + 2);
5 *(optr + 1) = *(iptr + 3);
```

or

```
1 int64_t *iptr = (int64_t *)ipt;
2 int64_t *optr = (int64_t *)opt;
3 ***optr = *(iptr + 1);
```

Soln:

The memory map of the

Address	Byte 0	Byte 1	Byte 2	Byte 3
ipt + 0x00:	0x00	0x10	0x20	0x30
ipt + 0x04:	0x40	0x50	0x60	0x70
ipt + 0x08:	0x80	0x90	0xA0	0xB0
ipt + 0x0C:	0xC0	0xD0	0xE0	0xF0

Part (a).

```
1 0xB0A0_9080, 0xF0E0_D0C0
```

Part (b).

```
1 0x0000_0040, 0xFFFF_FF80, r0 = 0x2000_1018, r1 = 0x2000_1040
```

Part (c).

```
1 0x0000_1000, 0xFFFF_B0A0, r0 = 0x2000_101E, r1 = 0x2000_1040
```

Part (d).

```
1 0x0000_3020, 0x0000_5040, r2 = 0x03, r3 = 0x0000_5040
```

Part (e). The solution is not unique. To save space, we have omitted the prototype for the functions below.

```
1 // Task 1
2 int32_t *ipter = (int32_t *)ipt;
3 *opt = *(ipter + 2);
4 *(opt+1) = *(ipter + 3);
5 ipter += 2;
6
7 or
8 int64_t *ipter = (int64_t *)ipt;
9 *(int64_t *)opt = +++ipter;
10
11 // Task 2
12 int8_t *ipt8 = (int8_t *)ipt;
13 ipt8 += 4;
14 *opt = (int32_t)*ipt8;
15 ipt8 += 4;
16 *(opt+1) = (int32_t)*ipt8;
17
18 // Task 3
19 int16_t *ipt16 = (int16_t *)ipt;
20 *opt = (int32_t)*ipt16;
21 int16 += 5;
22 *(opt+1) = (int32_t)*ipt16;
23 int16 += 2;
24
25 // Task 4
26 uint16_t *iptu16 = (uint16_t *)ipt;
27 *opt = (int32_t)*(iptu16 + i++);
28 *(opt+1) = (int32_t)*(ipt16 + i++);
```

```
:-
```